

Comparing Three Numerical Ways to Move a Two-Link Robot Arm

Group-3, Section C-2

2105161, 2105163, 2105173, 2105174, 2105176, 2105179

Dept. of Computer Science and Engineering, BUET

Dhaka, Bangladesh

Abstract

A two-link robot arm needs to move from one position to another using as little motor torque as possible, without going past its torque or speed limits. There is no direct formula for this; it has to be solved step by step with a numerical optimizer. We built and compared three well known ways of doing this: Single Shooting, Trapezoidal Collocation, and Hermite-Simpson Collocation, all using the same physics model of the arm. We tested all three on four scenarios of different difficulty, then improved the Trapezoidal solver with five separate code changes and measured what each change contributed on its own. The improved version is 22 to 41 times faster on the baseline scenario and returns the same answer on five of our six test scenarios. We also built an independent accuracy track: a trapezoidal-consistent interpolant for the motion between time steps, a convergence-order study that confirms the method is second order, and two stronger obstacle checks. Those obstacle checks uncovered a real defect in our own earlier results: a run the optimizer reports as a success can still drive both links straight through the obstacle, because the constraint was only ever checked on the hand and only at the time steps. This project follows the plan from our proposal: we reuse the collocation formulation described by Cardona-Ortiz and Arechavaleta [1] and the tutorials of Kelly [2, 3] and Tedrake [4], applied to a small two joint arm instead of a large industrial robot.

Code and Demo: [GitHub Repository](#) | [Demo Video](#)

1 Introduction and Base Paper Review

1.1 The Problem

Picture a robot arm with two rigid links, like a shoulder and an elbow, moving in a vertical plane under gravity. We want it to move from a starting pose to a target pose in a fixed amount of time, using as little total torque as possible, and never exceeding its torque or speed limits. There is no simple formula for this, because the motion has to obey the laws of physics at every single moment, and there are infinitely many moments to check. The usual way to make this solvable is to only look at a limited number of time points instead of every instant, and turn the problem into a system of equations and inequalities that a computer can search through. This is called direct transcription. There is more than one way to do this, and the choice affects speed, accuracy, and reliability a lot.

Our proposal set this up as an exploratory, side by side comparison, not as a new algorithm. We picked three well known transcriptions, Single Shooting, Trapezoidal Collocation, and Hermite-Simpson Collocation, and ran them on the exact same arm and the exact same tasks so we could see their differences clearly.

1.2 Base Paper and References

The sparsity-aware collocation formulation we follow comes from Cardona-Ortiz and Arechavaleta [1], who apply it to large, many jointed robots. We reuse their overall idea, but on a much smaller two joint arm, where every matrix and every constraint can be worked out and checked by hand. The specific defect and quadrature formulas we implement are the standard ones, taken from Kelly's tutorial [2, 3], which explains trapezoidal and Hermite-Simpson collocation with worked out diagrams, and Tedrake's course notes [4], which derive the Hermite-Simpson midpoint conditions from fitting a cubic curve through each time interval. These sources explain the same equations from different angles, and our code matches them.

1.3 What We Did

The project has three workstreams. The first built the shared model and the three-way benchmark; the second made the Trapezoidal solver fast; the third attacked its accuracy and its correctness. In short:

- Writing the physics model of the two-link arm from scratch (mass, gravity, and the coupling forces between the two joints).
- Implementing all three optimization methods (Shooting, Trapezoidal, Hermite-Simpson) on top of that same physics model.
- Designing six test scenarios of increasing difficulty, including one where the motors are deliberately too weak to lift the arm directly.
- Running all three methods on the benchmark scenarios and recording solve time, torque usage, and how many optimizer iterations each one needed.
- Building an independent physics check, using a completely separate and far more accurate simulator, to catch any solver that reports success without actually solving the real problem.
- Rewriting the Trapezoidal solver five separate times, each version adding exactly one speed improvement, so we could measure what each change contributed on its own instead of guessing.
- Deriving the interpolant that trapezoidal collocation actually implies between time steps, and using it to measure the method's convergence order and to check the obstacle constraint between time steps as well as at them.
- Replacing the hand-only obstacle check with an exact whole-arm check, after discovering that the hand-only version lets the links pass through the obstacle.

All of this is covered by 35 unit tests, which pass.

2 Methodology

2.1 Arm Dynamics

The state of the arm is $x = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]$: the two joint angles and how fast they are changing. The control is $u = [\tau_1, \tau_2]$: the torque applied at each joint. Standard rigid body physics for a two link arm gives us

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau,$$

where $q = [\theta_1, \theta_2]$ is just the two angles. In plain words: the torque you apply has to overcome the arm's own inertia, the extra forces that show up when both joints are moving at once, and gravity. The inertia part, $M(q)$, works out to

$$M_{11} = m_1 r_1^2 + I_1 + m_2 (l_1^2 + r_2^2 + 2l_1 r_2 \cos \theta_2) + I_2,$$

$$M_{12} = M_{21} = m_2 (r_2^2 + l_1 r_2 \cos \theta_2) + I_2,$$

$$M_{22} = m_2 r_2^2 + I_2.$$

With $h = m_2 l_1 r_2 \sin \theta_2$, the coupling forces and gravity work out to

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} -h(2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2) \\ h\dot{\theta}_1^2 \end{bmatrix},$$

$$g(q) = \begin{bmatrix} (m_1 r_1 + m_2 l_1)g_0 \cos \theta_1 + m_2 r_2 g_0 \cos(\theta_1 + \theta_2) \\ m_2 r_2 g_0 \cos(\theta_1 + \theta_2) \end{bmatrix}.$$

Rearranging for the accelerations gives the equation every method in this project actually calls,

$$\dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ M(q)^{-1}(u - C(q, \dot{q})\dot{q} - g(q)) \end{bmatrix}.$$

Both links are uniform rods with $l_i = 1$ m, $m_i = 1$ kg, $r_i = 0.5$ m and $I_i = \frac{1}{12}$ kg·m², and $g_0 = 9.81$ m/s². Since M is only a 2×2 matrix, its inverse can be written out directly instead of solving a linear system every time, using $\det M = M_{11}M_{22} - M_{12}^2$. The baseline solvers call a general linear solve here; the improved solver uses the closed form, and that shortcut turns out to matter a lot for speed later on.

The running cost used by all three methods is

$$\ell(x, u) = \|u\|^2 + w\|\dot{q}\|^2, \quad w = 10^{-3},$$

where the small velocity term is there only to keep the problem well posed.

2.2 Method 1: Single Shooting

Following the direct shooting setup described by Tedrake [4], the optimizer's only job here is to pick good torque values $u_0, \dots, u_{N-1}$. Each one is held *constant* across its interval (a zero-order hold). Once the optimizer has a guess for the torques, the resulting motion is simply simulated forward in time, one classical fourth order Runge-Kutta step per interval,

$$x_{k+1} = x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

with $k_1 = f(x_k, u_k)$, $k_2 = f(x_k + \frac{h}{2}k_1, u_k)$, $k_3 = f(x_k + \frac{h}{2}k_2, u_k)$, $k_4 = f(x_k + hk_3, u_k)$. The optimizer just needs the simulated end state to match the goal, $x_N(u) = x_f$. Each individual RK4 step is fourth order accurate, so the weakness of shooting is not its accuracy but its *conditioning*: x_N is built by chaining N integration steps together, so a tiny change to the very first torque value gets amplified through every step that follows. This is the conditioning problem Tedrake

describes for shooting methods on long or unstable trajectories, and it is the reason Shooting struggles later in our results.

2.3 Method 2: Trapezoidal Collocation

This time both the arm's state and its torque at every time step are decision variables the optimizer can adjust, so $z = [x_0, u_0, \dots, x_N, u_N]$ has $6(N+1)$ entries. Instead of simulating forward, physics is enforced by a consistency check between each pair of neighbouring time steps, using the trapezoid rule,

$$x_{k+1} - x_k = \frac{h}{2}(f(x_k, u_k) + f(x_{k+1}, u_{k+1})).$$

The total cost is calculated the same way, using the trapezoid rule on the running cost,

$$J = \sum_{k=0}^{N-1} \frac{h}{2}(\ell_k + \ell_{k+1}).$$

Because each check only ever involves two neighbouring time steps, an error at one point in time cannot spread across the whole trajectory the way it can in Shooting.

2.4 Method 3: Hermite-Simpson Collocation

This method adds one more point in the middle of every time interval and fits a cubic spline through each segment, matching the derivation in Tedrake [4] and the version used by Kelly [2, 3] and Betts [5].

We implement the *separated* form. This is worth stating precisely, because it is easy to describe the method as if the midpoint were computed from its neighbours, and that is not what the separated form does. The midpoint state x_c and the midpoint control u_c are *independent decision variables*, exactly like the ones at the grid nodes. So with $2N+1$ points in total the optimizer sees $6(2N+1) = 12N+6$ unknowns. The midpoint state is then tied to its neighbours by an equality constraint rather than substituted away,

$$x_c - \frac{1}{2}(x_k + x_{k+1}) - \frac{h}{8}(f_k - f_{k+1}) = 0,$$

which is what the cubic curve gives at the midpoint once the end-point states and slopes $f_k = f(x_k, u_k)$, $f_{k+1} = f(x_{k+1}, u_{k+1})$ are known. The midpoint control u_c carries no such constraint at all: the optimizer is free to choose it, so the control is effectively piecewise quadratic rather than piecewise linear. Physics is then enforced over the whole interval with Simpson's rule,

$$x_{k+1} - x_k = \frac{h}{6}(f_k + 4f(x_c, u_c) + f_{k+1}).$$

Writing these as two separate equations, instead of one combined condition with x_c and u_c eliminated, is what Kelly and Betts call the separated form; the compressed form substitutes them in and carries $6(N+1)$ unknowns instead. The cost uses Simpson's weighting to match,

$$J = \sum_{k=0}^{N-1} \frac{h}{6}(\ell_k + 4\ell_c + \ell_{k+1}).$$

This roughly doubles the number of unknowns per interval, but the error now shrinks with the fourth power of the step size instead of the second.

2.5 Speeding Up the Trapezoidal Solver

We changed the Trapezoidal solver five separate times, one change at a time, without ever touching the equations above, so the mathematical problem stays identical and only the way it is handed to the optimizer changes.

S1: Fix the start and end directly. Instead of two extra equations telling the optimizer “the first and last state must equal these exact values,” we set the lower and upper bound of those variables to the same number, so the optimizer cannot move them at all. This removes 8 equality constraints.

S2: Compute the physics for every point at once. The consistency checks need $f(x_k, u_k)$ at every time step. Instead of computing this one step at a time in a Python loop, we compute all of them together in one batch of array operations, using the closed-form 2×2 inverse of $M(q)$ instead of a general linear solve at every single point.

S3: Tell the optimizer which variables actually matter to each equation. The optimizer needs to know how every equation changes as every variable changes. The default way to find this out is to nudge one variable at a time and see what happens,

$$\frac{\partial F}{\partial z_c} \approx \frac{F(z + \epsilon e_c) - F(z)}{\epsilon},$$

which costs one full recalculation per variable, that is $6(N+1)$ of them. But since each consistency check only involves two neighbouring time steps, variables that never appear together in the same equation can safely be nudged at the same time: we can still tell which change caused which effect. A greedy graph colouring groups the variables this way, and it always finds exactly 12 groups (6 variables per node $\times$ {even nodes, odd nodes}) no matter how many time steps we use. We also supply the objective’s gradient in closed form, since the cost is quadratic and its derivative is exact.

S4: Rescale the numbers. Angles are roughly the size of 1, joint speeds run up to $\dot{q}_{\max}$ (15 rad/s), and torques run up to $\tau_{\max}$ (30 N·m on the baseline). We rescale every variable to a similar range, $y = z/s$, building s from $\dot{q}_{\max}$ and $\tau_{\max}$, and we rescale the cost by $h \tau_{\max}^2$ to match. We found that rescaling the variables alone, without also rescaling the cost, actually made the optimizer’s job harder, because the cost then curves about $900\times$ more sharply in the rescaled torque direction than the optimizer’s starting assumption expects.

S5: Start from an easier version of the problem first. Instead of solving directly at the full number of time steps N , we repeatedly halve N while it stays at least 8, solve the coarsest grid first, stretch that answer onto the next grid up by linear interpolation to use as the starting guess, and repeat until we reach N . For $N = 40$ the ladder is $10 \rightarrow 20 \rightarrow 40$; for $N = 80$ it is $10 \rightarrow 20 \rightarrow 40 \rightarrow 80$. Solving a sequence of easy problems this way ends up faster overall than solving the hard problem directly from a plain straight line guess. All reported warm-start times and iteration counts are totals that *include* the coarse solves.

2.6 Getting the Motion Between the Time Steps Right

Everything above only pins down the motion *at* the time steps. Several questions – how accurate is the method really, and is the arm actually clear of the obstacle – depend on what happens *between*

them, so we derived the interpolant that trapezoidal collocation actually implies.

Trapezoidal collocation assumes $\dot{x} = f$ varies linearly across an interval. Integrating that assumption once gives a parabola in the state and a straight line in the control: for $t = t_k + \sigma$ with $0 \leq \sigma \leq h$,

$$x(t) = x_k + f_k \sigma + \frac{\sigma^2}{2h}(f_{k+1} - f_k), \quad u(t) = u_k + \frac{\sigma}{h}(u_{k+1} - u_k).$$

At $\sigma = h$ this returns $x_k + \frac{h}{2}(f_k + f_{k+1})$, which is exactly x_{k+1} whenever the defect is zero, so the parabola is continuous across nodes and its slope at each node is $f(x_k, u_k)$ by construction. This is the natural interpolant for the method; drawing straight lines between nodes instead, as our first version did, is inconsistent with the very assumption the defects encode.

Two things build directly on it. Setting $\sigma = h/2$ gives the interval midpoint in closed form,

$$x_{\text{mid}} = x_k + \frac{h}{8}(3f_k + f_{k+1}), \quad u_{\text{mid}} = \frac{1}{2}(u_k + u_{k+1}),$$

which lets us evaluate any path constraint at the midpoints as well as at the nodes. Each midpoint row depends only on nodes k and $k+1$, the same pattern as the defects, so the S3 colouring still gives 12 groups and the extra constraints cost almost nothing per Jacobian. And sampling the parabola densely inside every interval gives an honest clearance measurement that a node-only check cannot provide.

2.7 Checking the Whole Arm, Not Just the Hand

Our first obstacle constraint kept only the end-effector outside the circle, $\|p_{ee}(q_k) - p_{\text{obs}}\|^2 \geq r^2$, enforced at every node. Neither of those restrictions is safe. The stronger version enforces one constraint per link: the exact squared distance from the obstacle centre to the link *segment* – the projection of the centre onto the segment, clamped to the segment’s ends – minus $(r + 5 \text{ mm})^2$. Because link 2 ends at the hand, the whole-arm check subsumes the hand-only one. Using the exact segment distance rather than sampling points along the link matters: sample points spaced 0.25 apart would let the circle dig about 0.03 m into the gap between them.

2.8 Checking Against Real Physics

Just because a method’s own equations are satisfied does not mean the motion is actually physically correct in between the time steps it checked. To verify this independently, we take the optimizer’s final torque schedule, interpolate it in time, and feed it into a completely separate, very accurate physics simulator (an adaptive Runge-Kutta method called DOP853, with relative and absolute tolerance 10^{-8}), unrelated to the RK4 stepper used inside Shooting. We then measure the biggest gap, both in the arm’s state and in where the hand actually ends up, between this independent replay and what the optimizer originally claimed would happen.

One detail decides whether this comparison is fair. The replay has to reconstruct the torque between the time steps, and the three methods do not assume the same thing there: Trapezoidal and Hermite-Simpson assume a control that varies continuously between nodes, while Shooting assumes each torque is *held constant* across its interval. Replaying every method with the same linear

interpolation therefore hands Shooting a control signal it never asked for, and charges it for the difference. Section 4.1 reports both conventions so this effect can be separated from the genuine conditioning problem.

3 Implementation and Experiments

3.1 Software

Everything is written in Python, using NumPy for array math, SciPy’s [6] SLSQP optimizer and DOP853 integrator, and Matplotlib for plots. The physics model, the three solvers, the test scenarios, the accuracy helpers, and the metrics and physics-check code are each kept in their own files, all built around one shared problem description (start pose, goal pose, duration, torque and speed limits, and any obstacles) and one shared result format (time grid, state trajectory, torque trajectory, cost, solve time, and iteration count), so every method can be compared fairly on exactly the same terms. A suite of 35 unit tests covers the dynamics, the solvers, every speed flag individually and in combination, the interpolant’s continuity and slope conditions, the sparse Jacobian against a dense finite-difference reference, and the obstacle geometry. The full source code is available in our [GitHub repository](#), and a video demonstration of the project is available on [Project Demo](#).

3.2 Scenarios

- **Baseline** (rest_to_rest): horizontal $[0, 0]$ to upright $[\pi/2, 0]$, 1.0 s, $\tau_{\max} = 30$ N·m.
- **Obstacle**: $[-\pi/4, 0]$ to $[\pi/4, 0]$, 1.2 s, $\tau_{\max} = 40$ N·m, with a circular keep-out zone of radius 0.35 m centred at (1.20, 0) m.
- **Swing-up** (swing_up): hanging straight down $[-\pi/2, 0]$ to straight up $[\pi/2, 0]$, 2.0 s, $\tau_{\max}$ only 6 N·m against a peak stall torque of about 14.7 N·m, so the arm cannot lift directly and has to swing back and forth first to build up momentum.
- **Fast maneuver** (high_speed): $[-\pi/3, \pi/4]$ to $[\pi/2, -\pi/3]$ in only 0.45 s, $\tau_{\max} = 80$ N·m, $\dot{q}_{\max} = 25$ rad/s, where the velocity-squared coupling forces stop being small enough to ignore.

Two further scenarios are used only in the speed study: **under-torqued** (the same inversion as swing-up but in 1.2 s with $\tau_{\max} = 10$ N·m) and **reversal** ($[0, 0]$ to $[\pi, 0]$ in 1.0 s with $\tau_{\max} = 45$ N·m, sweeping through the singular stretched-out pose).

3.3 Solver Settings and How We Measured the Speedups

The three-way benchmark runs all three solvers through one shared harness with identical settings: SLSQP, at most 400 iterations, and a cost tolerance of 10^{-5} . The starting guess is always a straight line between the start and goal poses, with just enough torque to hold the arm’s weight against gravity. The Trapezoidal speed study is a separate harness and uses the solvers’ own defaults, 400 iterations and a tolerance of 10^{-6} ; the convergence study tightens this to 10^{-10} with up to 1000 iterations so that the optimizer’s stopping tolerance cannot contaminate a discretization-error measurement. To measure the speedups, we turned the five improvements on one

at a time on the baseline scenario, then repeated the comparison at several different numbers of time steps, and finally across all six scenarios, to make sure the speedup was not a fluke of one particular setting. All timings are the median of 3 runs on one machine; the original solver at $N \geq 80$ was timed once because of how slow it is.

4 Results and Discussion

4.1 Baseline Scenario

Table 1: Baseline results (20 time steps, 1.0 s horizon).

Metric	Shoot.	Trap.	H-S
Solve time (s)	6.37	3.14	12.26
Optimizer iterations	53	39	54
Total torque effort	282.9	223.4	217.8
Peak torque (N·m)	25.1	26.5	28.6
Max drift when replayed (m)	1.902	0.120	0.280

All three report success. Hermite-Simpson uses the least torque overall, about 23 percent less than Shooting, thanks to its richer, curve based description of the motion. Trapezoidal is the quickest to solve.

The independent physics check appears to tell a dramatic story: Shooting’s own simulator says it landed exactly on target, yet the replay lands 1.77 m away at the final time. We originally reported this as evidence that Shooting produces a trajectory which does not follow real physics. **That conclusion does not survive scrutiny**, and the reason is the measurement convention flagged in Section 2.8. The replay reconstructs the torque between nodes by linear interpolation, which is what the collocation methods assume, but Shooting assumes a zero-order hold. Re-running every solution under *both* reconstructions separates the two effects.

Table 2: Final end-effector drift when each solution is replayed under both torque reconstructions. Bold marks the convention that solver actually assumes.

Scenario	Method	Linear	Zero-order hold
Baseline	Shooting	1.771 m	0.234 m
	Trap.	0.071 m	2.973 m
	H-S	0.280 m	1.793 m
Obstacle	Shooting	1.317 m	0.034 m
	Trap.	0.267 m	2.995 m
	H-S	0.405 m	3.594 m

The pattern is unambiguous and symmetric: *every* method is accurate under its own control convention and looks badly wrong under the other one. Shooting’s baseline drift falls from 1.771 m to 0.234 m once it is replayed the way it was solved, and on the obstacle scenario it falls to 0.034 m, which is an order of magnitude *better* than Trapezoidal’s 0.267 m. Conversely, replaying Trapezoidal under a zero-order hold makes it look catastrophic at 2.973 m.

What this measures, therefore, is mostly the mismatch between the assumed and the replayed control, not the quality of the transcription. The drift columns in Tables 1, 3 and 4 are all computed with linear reconstruction, so they systematically penalise Shooting and should not be read as a physics-fidelity ranking. We keep them for consistency with the rest of the study and because they are the numbers our benchmark harness produces, but the claim they were originally used to support has been withdrawn.

Shooting’s real weaknesses are visible elsewhere and do not depend on this convention: it uses about 23 percent more control effort than Hermite-Simpson here, it needs nearly twice as many iterations as Trapezoidal on the obstacle scenario, and it fails outright on swing-up. Its extreme sensitivity to how the torque is reconstructed is itself a genuine practical drawback for an open-loop plan, but it is a different and much narrower claim than “the trajectory does not follow real physics.”

4.2 Obstacle Scenario

Table 3: Obstacle avoidance results (25 time steps, 1.2 s horizon), hand-only node check.

Metric	Shoot.	Trap.	H-S
Solve time (s)	29.8	11.7	32.9
Iterations	143	80	86
Max drift replayed (m)	1.317	0.267	0.405

In Shooting, the obstacle rule has to be checked along the whole simulated path, so small simulation wobbles make it jump around unpredictably between attempts, which nearly doubles the number of iterations needed. In the collocation methods, the same rule is checked directly against each time step’s own known position, which is a much steadier calculation. That iteration count, not the drift row, is the meaningful difference here: as Table 2 shows, Shooting’s drift on this scenario is 0.034 m once it is replayed under its own control convention, better than either collocation method.

These runs are not actually collision-free, and that is the most important result in this section. Section 4.6 shows that the hand-only, node-only constraint used here is satisfied to the letter while the arm still passes through the obstacle. Table 3 should be read as a comparison of how the three methods handle a nonlinear path constraint, not as evidence that any of them avoided the obstacle.

4.3 Swing-Up

On the 2.0 s swing-up with $\tau_{\max} = 6 \text{ N}\cdot\text{m}$, Shooting does not converge. It exhausts its 400 iteration budget after about 1420 s and stops with a constraint violation of 1.4×10^{-3} ; it gets close to the target but never satisfies the terminal condition, and SLSQP reports failure. The arm has to swing backward first to build up momentum, and over that two second motion a tiny change in the very first torque value is amplified so much by the time it reaches the end that the optimizer can no longer establish a reliable descent direction. Both collocation methods handle this scenario without trouble, reaching essentially the same optimum (cost 60.55 for Trapezoidal, 60.51 for Hermite-Simpson) with zero terminal error, since their

consistency checks only ever look at two neighbouring time steps at a time.

4.4 Fast Maneuver

Table 4: Fast maneuver results (25 time steps, 0.45 s horizon).

Metric	Shoot.	Trap.	H-S
Solve time (s)	24.4	55.2	101.1
Optimizer iterations	58	237	134
Total torque effort	1336.6	1214.4	1208.4
Peak torque (N·m)	80.0	74.7	77.0
Max drift when replayed (m)	1.849	0.020	0.023
Final drift when replayed (m)	1.718	0.009	0.023

At 0.45 seconds the coupling forces between the two joints are no longer small, and this is the scenario where we expected Hermite-Simpson’s higher order to pull clearly ahead of Trapezoidal at the same number of time steps. **It does not.** At $N = 25$ the two collocation methods track real physics about equally well, and Trapezoidal is in fact slightly closer (0.020 m versus 0.023 m maximum drift, 0.009 m versus 0.023 m at the final time). Hermite-Simpson wins only on control effort, and only by 0.5 percent, for 1.8 times the solve time.

Shooting’s 1.8 m drift row should be discounted for the reason given in Section 4.1: it is replayed under a control convention it did not assume, and we did not re-run this scenario under the zero-order hold. What the scenario does show is that Shooting needs 80 N·m of torque, the full actuator limit, where both collocation methods complete the same maneuver with headroom to spare and about 10 percent less effort.

We had originally claimed a Trapezoidal-versus-Hermite-Simpson accuracy gap here on the strength of theory alone; measuring it showed the claim was not supported at this grid size. Hermite-Simpson’s fourth-order advantage should appear on much coarser grids, where the second-order method still has visible error; demonstrating that would need a sweep over N in the range 8 to 15, which we did not run.

4.5 Speeding Up the Trapezoidal Solver

Table 5: Solve time as each change is added (40 time steps, baseline scenario). Every row gives the same answer to within 1.4×10^{-7} relative cost.

Version	Time (s)	Speedup	Iters
Original	21.54	1.0×	46
+ S1 fix start/end directly	20.32	1.1×	46
+ S2 batch the physics	7.05	3.1×	46
+ S3 smarter derivative pattern	3.44	6.3×	46
+ S4 rescale the numbers	2.12	10.2×	46
+ S5 solve easy version first	0.98	22×	65

4.5.1 What Each Change Contributed. Batching the physics computation gives the single biggest jump on its own, which tells us

the original slowdown was mostly Python doing things one at a time, not the underlying math being hard. Telling the optimizer the derivative pattern roughly doubles the gain again, by cutting the $6(N+1) = 246$ recalculations per Jacobian at $N = 40$ down to 12. Solving the easy version first uses more total iterations added up across all the grid sizes (65 against 46), but still wins on wall-clock time, because most of those iterations run on cheap coarse grids.

Table 6: Solve time vs. number of time steps N , baseline scenario.

N	Orig.	S1-S4	All 5	S1-S4	All 5
10	2.55 s	0.18 s	0.18 s	14.6×	14.4×
20	7.01 s	0.46 s	0.33 s	15.3×	21.5×
40	21.30 s	2.15 s	0.98 s	9.9×	21.7×
80	119.87 s	9.90 s	2.95 s	12.1×	40.6×

4.5.2 Does It Still Work at Different Sizes? The advantage gets bigger, not smaller, as we add more time steps. The original solver gets slower per iteration as the number of variables grows, but the smarter derivative pattern always stays at 12 groups no matter how many time steps we use. At $N = 10$ the warm start has no effect, because halving 10 already falls below the coarse-grid floor of 8, so there is no coarse level to solve.

Table 7: All six scenarios at 30 time steps: original vs. fully improved solver.

Scenario	Orig.	Impr.	Speedup	Answer
Baseline	14.53 s	0.61 s	23.8×	same
Obstacle	28.78 s	3.74 s	7.7×	29% cheaper
Fast maneuver	34.63 s	0.81 s	42.9×	same
Under-torqued	12.73 s	0.42 s	30.3×	same
Reversal	93.54 s	1.66 s	56.4×	same
Swing-up	19.23 s	0.83 s	23.1×	same

4.5.3 Does It Work on Every Scenario? The obstacle scenario gains the least, since its obstacle constraint was never one of the parts we vectorized, so it still runs node by node in Python. It is also the one scenario where the improved solver returns a genuinely *different* answer: a 29 percent cheaper path (cost 239.2 against 335.3). Both are valid local minima of the problem as posed. The reversal scenario behaves the same way when S5 is left off: S1-S4 alone land in a different valley (cost 1019.8 against 566.3), and we confirmed 1019.8 is a real local minimum rather than a bug by restarting the original solver from it, which accepts it after one iteration. Warm starting removes that sensitivity, because the coarse solve picks the valley and the fine solves only refine it. This is the main reason we recommend the warm-started configuration as the default: it is both the fastest and the most reproducible.

4.5.4 Making Sure the Math Is Still Correct. We checked how fast the error shrinks as we add more time steps, by fitting a line through $\log(\text{error})$ against $\log(h)$ for $N = 10, 15, 20, 30, 40, 80, 160$, using the

Table 8: Fitted convergence order (slope of log error vs. log h).

Scenario	Error measure	all N	$N \geq 20$
Baseline	max state error	1.90	2.04
Baseline	final drift	2.81	2.48
Fast maneuver	max state error	1.76	1.97
Fast maneuver	final drift	2.56	2.17

interpolant of Section 2.6 so that the error measured is the method’s own, not an artefact of drawing straight lines between nodes.

The max state error, fitted over $N \geq 20$, gives order 2.04 and 1.97, matching the theoretical $O(h^2)$ almost exactly; from $N = 40 \rightarrow 80 \rightarrow 160$ the error falls by $4.0\times$ and $3.9\times$ per doubling in both scenarios, against the $4\times$ the theory predicts. Coarse grids ($N \leq 15$) are not yet in the asymptotic regime, which is why the headline fit excludes them. Final drift is the noisier measure (2.48 and 2.17) because it looks at a single point in time, where errors from different parts of the motion can partly cancel; max state error is the more reliable indicator of order. Throughout, the maximum constraint violation stays below 9×10^{-13} , at least eight orders of magnitude below the smallest error measured, so the optimizer’s own tolerance is not distorting the result.

4.6 Correctness of the Obstacle Constraint

4.6.1 The Hand-Only Node Check Is Not Safe. Measuring clearance densely along the interpolant, rather than only at nodes, and for both links rather than only the hand, shows that our original obstacle runs are not collision-free at all. At $N = 30$ the solver succeeds, the hand sits exactly on its 5 mm safety margin at every node, and yet:

- both links reach a minimum clearance of -0.150 m, with a link inside the obstacle at 5 of the 31 nodes;
- between nodes, even the hand penetrates 0.015 m.

The constraint is satisfied exactly as written. The trouble is that it was the wrong constraint, in two independent ways: it watched one point on a body that has length, and it watched it only at the instants the optimizer happened to be looking.

4.6.2 Checking the Whole Arm. Switching on the exact per-link segment constraint makes the *original* obstacle infeasible, and correctly so: the arm must pass through $\theta_1 = 0$, where link 1 lies along the x -axis just 0.2 m from the obstacle’s centre, so no whole-arm-feasible path exists. We therefore chose a new obstacle for this comparison, at $(1.5, -0.3)$ with radius 0.3 m, which is out of link 1’s reach ($1.53 - 0.3 = 1.23$ m $> l_1 = 1$ m) but well inside link 2’s sweep, so the problem stays solvable while still exercising the check.

From a cold start, every hand-only run swings the hand around the outside of the obstacle while link 2 cuts through its centre: -0.300 m is the full radius. The whole-arm check produces a collision-free path in all six configurations we tried (three grid sizes, cold and warm start), with $+0.23$ m of clearance, by folding the elbow and passing inside the obstacle instead. The hand-only problem turns out to have two local minima, a colliding “swing around” path costing about 400 and the folded path costing about 171; warm starting happens to find the safe one, but by luck, not because anything checks the links. The whole-arm constraint removes the

Table 9: New obstacle: minimum clearance of any part of the arm, measured densely along the trajectory. Negative means collision.

N	Start	Check	Cost	Min clearance
20	cold	hand only	417.2	-0.300 m
20	cold	whole arm	172.5	+0.230 m
30	cold	hand only	414.8	-0.300 m
30	cold	whole arm	171.4	+0.230 m
40	cold	hand only	394.7	-0.299 m
40	cold	whole arm	171.0	+0.230 m

colliding minimum entirely, so the outcome no longer depends on the starting guess.

4.6.3 Checking Between the Time Steps. Corner cutting is the second, independent failure. Enforcing the same constraint at the interval midpoints as well as the nodes costs nothing in Jacobian structure, because a midpoint row depends on the same two nodes a defect row does.

Table 10: Original obstacle, hand-only check: worst hand penetration between nodes, with and without the midpoint constraint.

N	Midpoints off	Midpoints on	Cost (on)
8	-0.302 m	-0.333 m	253.5
10	-0.261 m	-0.156 m	322.7
15	-0.230 m	-0.013 m	300.7
20	-0.126 m	-0.003 m	304.2
30	-0.015 m	+0.002 m	297.2

At every grid size the hand sits exactly on the margin at the nodes, so “all constraints satisfied” is true, and yet it dives up to 0.30 m into an obstacle of radius 0.35 between two of them. At $N = 15$ the hand simply jumps from one side of the obstacle to the other inside a single interval. Adding the midpoints fixes most of this for $N \geq 15$ and produces the first genuinely collision-free run at $N = 30$. It does not help at $N = 8$: with intervals that long the path can pass between a node and a midpoint, and halving the spacing between checked points is not the same as guaranteeing the curve. Safety also costs effort – at $N = 30$ the cost rises from 239.2 to 297.2, about 24 percent – because the cheap solution was only cheap by virtue of cutting through the obstacle.

4.7 Summary Comparison

5 Conclusion and Future Work

5.1 Key Takeaways

- (1) All three methods call the exact same physics function $f(x, u)$, so every difference in how they perform comes purely from how each one arranges its equations over time, not from different physics.
- (2) Optimizer success and actual physical correctness are two different things. Our obstacle runs satisfied every constraint the optimizer was given, to the last decimal place, while

Table 11: Quick comparison of the three methods.

Property	Shooting	Trap.	H-S
Unknowns	$2N$	$6(N+1)$	$12N+6$
Checks physics	Whole trajectory	2 neighbours	2 neighbours plus midpoint
Accuracy	$O(h^4)$ per step, but ill-conditioned	$O(h^2)$, measured 2.04	$O(h^4)$
Fails on swing-up?	yes	no	no

driving both links straight through the obstacle: the constraint watched one point on a body that has length, and watched it only at the instants the optimizer happened to look. “All constraints satisfied” is a statement about the model you wrote down, not about the world.

- (3) A measurement convention can dominate, and even invert, the quantity being measured. We initially read Shooting’s 1.9 m replay drift as proof that it produced an unphysical trajectory. Almost all of that number came from reconstructing its torque by linear interpolation when it had assumed a zero-order hold; under its own convention the drift is 0.234 m on the baseline and 0.034 m on the obstacle scenario, where it beats both collocation methods. The ranking we drew from that metric did not survive the correction, and we withdrew the claim. The lesson we would keep is to check that a comparison metric is not quietly measuring your own choice of convention before you build a conclusion on it.
- (4) The five speed changes each fix a different bottleneck (Python overhead, wasted recalculations, mismatched number scales, a poor starting guess), and stacking them together gives 22 to 41 times faster solves on the baseline while returning the same answer on five of our six scenarios. On the sixth, the improved solver finds a different and 29 percent cheaper local minimum.
- (5) A change applied only halfway can backfire. Rescaling the variables without also rescaling the cost made the optimizer need 115 iterations instead of 46, a reminder to test each change on its own rather than assume it helps.
- (6) Theory is a hypothesis, not a result. We expected Hermite-Simpson to beat Trapezoidal on the fast maneuver at equal grid size; measured, it does not, at $N = 25$.

5.2 Limitations

- All timings come from one machine, taking the median of 3 runs.
- The reality check reconstructs every method’s torque by linear interpolation, which matches what the collocation methods assume but not what Shooting assumes. Table 2 quantifies this for the baseline and obstacle scenarios; we did not repeat it for the fast maneuver or swing-up, so Shooting’s drift figures there carry the same upward bias and should not be quoted. A fully fair benchmark would replay each method under the control representation it was

solved with, which would mean reporting two drift columns throughout rather than one.

- SLSQP can get stuck in a locally good but not globally best answer, which is why the obstacle and reversal scenarios can land on a different (usually cheaper) solution after the speed changes.
- The physics check is a one-time open-loop replay and does not model a feedback controller correcting the arm as it moves.
- The whole-arm and midpoint obstacle checks are implemented only in the improved Trapezoidal solver. The three-way benchmark still uses the original hand-only node check, so Table 3 does not represent collision-free motion. Moving the benchmark over would require all three solvers to switch to the new obstacle together, since the old one is infeasible under the whole-arm check.
- The midpoint check halves the spacing between checked points; it does not certify the whole curve, and it demonstrably fails at $N = 8$.
- The obstacle constraint was never vectorized, so the obstacle scenario benefits least from the speed work.

5.3 Future Work

- Run the coarse-grid sweep ($N = 8$ to 15) that would show Hermite-Simpson's fourth-order advantage over Trapezoidal, which our $N = 25$ comparison was too fine to reveal.

- Move the whole-arm and midpoint obstacle checks into all three solvers so the headline benchmark reports genuinely collision-free trajectories.
- Replace the midpoint check with a certified one, for example by bounding the interpolant over each interval instead of sampling it.
- Replay each method under the control representation it was actually solved with, and report both columns, so the physics-fidelity comparison stops measuring the reconstruction convention.
- Vectorize the obstacle constraint the same way the defects were vectorized.
- Let the time steps automatically get smaller in parts of the motion that change quickly, instead of spacing them evenly.
- Try the same setup on an arm with 6 or 7 joints, closer to the scale Cardona-Ortiz and Arechavaleta [1] originally intended.

References

- [1] D. Cardona-Ortiz and G. Arechavaleta, "Trajectory optimization for highly articulated robots based on sparsity-free local direct collocation," *Int. J. Appl. Math. Comput. Sci.*, vol. 35, no. 4, pp. 577–589, 2025. doi:10.61822/amcs-2025-0041.
- [2] M. Kelly, "An introduction to trajectory optimization: How to do your own direct collocation," *SIAM Review*, vol. 59, no. 4, pp. 849–904, 2017.
- [3] M. Kelly, "Trajectory optimization tutorials," <https://www.matthewpeterkelly.com/tutorials/trajectoryOptimization/>, accessed 2026.
- [4] R. Tedrake, "Underactuated Robotics, Chapter 10: Trajectory Optimization," MIT, 2024. <https://underactuated.mit.edu/trajopt.html>.
- [5] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd ed. SIAM, Philadelphia, PA, 2010.
- [6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.